

Project Plan: Peer-to-Peer Distributed File Sharing System

Interim implementation status

The confirmed interim scope is **multi-tracker synchronization plus all user/group commands**. These are implemented under `include/` and `src/`, with a root Makefile, persistent recovery, a CLI client, and integration tests. See [README.md](#) for the actual protocol, configuration, guarantees, and run/test instructions, and [REPORT.md](#) for design rationale. File-transfer sections below remain the final-submission plan.

1. Goal and priorities

Build a command-line file sharing system in C++ in which users create groups, share files with group members, and download verified file pieces from multiple peers concurrently. Exactly two trackers manage metadata; clients transfer all file bytes directly to each other.

Priority order: correct commands and permissions → correct file transfer and hashes → tracker synchronization and recovery → concurrency and failure handling → performance improvements.

This plan is based on [AOS Assignment 3](#) and [Bram Cohen's BitTorrent paper](#). The assignment is the source of requirements; the paper supplies selected algorithm ideas.

Dates

Milestone	Deadline from assignment	Planned outcome
Interim	September 16, 2026, 11:59 PM	Multi-tracker synchronization and all user/group commands, implemented and tested
Final	September 26, 2026, 11:59 PM	Complete implementation, tests, README, technical report, and ZIP

The interim scope above was confirmed by the user. Schedule below starts September 15 and reserves the final day for packaging and fixes.

2. Requirements checklist

Requirement	Implementation approach	Acceptance evidence
Exactly two trackers	Same tracker executable, selected by tracker number	Both processes run from one configuration file
Accounts and sessions	Register/login/logout; session identifies caller	Valid/invalid login and logout tests
Groups and permissions	Owner, members, pending join requests	Unauthorized actions rejected
File publication and discovery	Group-scoped metadata and live peer registry	Group members discover available files and peers
512 KB pieces	Use $512 * 1024 = 524288$ bytes; final piece may be shorter	Boundary-size tests
Piece and whole-file SHA1	Streaming hashing and verification	Deliberate corruption is detected and retried
Files up to 1 GB	Bounded buffers; offset-based file I/O	Large-file transfer with recorded memory use

Requirement	Implementation approach	Acceptance evidence
Parallel peers per file	Multiple workers reserve different pieces	Logs show overlapping transfers from two seeders
Concurrent files per client	Download manager schedules multiple jobs	Two files make progress simultaneously
TCP custom protocol	Explicit framing and exact I/O loops	Fragmented/coalesced message tests
Tracker synchronization/recovery	Replicated mutation log and reconnect synchronization	Failure, restart, partition, and convergence tests
Clear download status	Exact completed format plus ongoing progress	[C] [group_id] filename
Resource and thread safety	Bounded workers, locking, scoped descriptor ownership	Stress tests and sanitizer checks
C/C++ and standard calls/libraries only	POSIX sockets/files, standard C++ containers and threads	Build with standard g++; dependency audit
Submission and documentation	Required source directories, README, separate report	Clean extraction/build/demo

Do not use `std::filesystem`, database libraries, torrent libraries, `system()`, `exec*()`, or `popen()` in the implementation. Do not assume OpenSSL is permitted: implement a small streaming SHA1 module using standard C++ and verify it against known test vectors. SHA1 is required for file integrity; do not describe it as a suitable password security design.

3. Architecture

flowchart LR

```

T1[Tracker 1: metadata] <-->|TCP replication| T2[Tracker 2: metadata]
A[Client A: CLI + peer server] <-->|Commands / discovery| T1
B[Client B: CLI + peer server] <-->|Commands / discovery| T2
C[Client C: CLI + peer server] <-->|Commands / discovery| T1
A <-->|Piece requests / bytes| B
B <-->|Piece requests / bytes| C
A <-->|Piece requests / bytes| C

```

Tracker responsibilities

- Authenticate users and validate each command's permissions.
- Maintain groups, pending requests, file hashes, and peer endpoints.
- Return metadata and candidate peers to authorized downloaders.
- Synchronize metadata mutations and recover after downtime.
- Handle its console `quit` command and close connections cleanly.

Client responsibilities

- Run the CLI and a listening peer server at the same time.
- Maintain a tracker connection with retry/failover handling.
- Hash and publish local files without sending file contents to trackers.
- Manage concurrent downloads, piece verification, retries, and progress.
- Serve locally verified pieces while downloads continue.
- Stop serving when logged out or when the corresponding share is stopped.

Keep the paths separate: tracker traffic carries control information; peer traffic carries file data. An established download should continue from known peers during a tracker outage. Operations needing fresh discovery or authorization should report temporary unavailability.

4. Implementation structure

Use C++17, POSIX socket/file APIs, `std::thread`, `std::mutex`, and `std::condition_variable`. Use `open`, `fstat`, `pread`, `pwrite`, `ftruncate`, and `close` for file access. Set a large-file build option where needed and use 64-bit sizes/offsets.

```
include/
  common/          # protocol.hpp, net.hpp; future sha1.hpp
  tracker/         # state.hpp, server.hpp
  client/          # cli.hpp; future peer/download headers
src/
  common/          # protocol.cpp, net.cpp; future sha1.cpp
  tracker/         # main.cpp, server.cpp, state.cpp, journal.cpp
  client/          # main.cpp, cli.cpp; future peer/download modules
tests/
README.md
REPORT.md
tracker_info.txt   # Two endpoint lines
Makefile           # Builds both components
Plan.md
```

The user's requested layout supersedes the initial directory proposal: all `.hpp` files go under `include/`, implementations under `src/`, and functionality gets its own subdirectory. The root Makefile builds both components. Adapt the final submission wrapper if the course requires top-level tracker/client directories; preserve this internal source layout.

Build root executables `./tracker` and `./client` and run from the project root:

```
./tracker tracker_info.txt 1
./tracker tracker_info.txt 2
./client 127.0.0.1:6001 tracker_info.txt
```

Proposed configuration format: two lines, each containing IP PORT; tracker numbers are 1 and 2. Document these assumptions. Development flags: `-std=c++17 -Wall -Wextra -Wpedantic -pthread -g`; add optimization for the final build after correctness checks.

5. State model and invariants

Tracker state

Record	Fields
User	User ID and credential representation
Session	Opaque token, user ID, client endpoint, generation, liveness
Group	Group ID, owner ID, member set, pending-request set, current members in acceptance order
File metadata	Group ID, display name, size, piece size/count, whole SHA1, piece SHA1 vector
Share	Group/file identity, user/session identity, peer endpoint, active state
Replication event	Origin tracker, origin sequence, logical timestamp, request ID, actor, operation and payload

Treat credentials as a separate concern from file hashes: keep them out of logs and error messages, and document the authentication/storage limitations of the educational TCP system. Do not invent a password scheme by reusing the file SHA1 routine.

Client state

Record	Fields
Local file	File identity, local path, file descriptor owner, verified bitset, active group shares
Download job	Metadata, destination/temp path, piece states, peer bitfields, counters, status, mutex
Piece state	MISSING, IN_FLIGHT, or VERIFIED; assigned peer, attempt number, deadline
Peer state	Endpoint, available-piece bitfield, failures, cooldown, active requests

Invariants to enforce in code

1. Trackers never store or relay file payload bytes.
2. Only authorized group members publish files or obtain download metadata.
3. Only the group owner lists/accepts requests.
4. A piece becomes shareable only after SHA1 verification and successful disk write.
5. A piece has at most one active owner/request in the baseline scheduler.
6. A file becomes completed only after all pieces and the final whole-file SHA1 pass.
7. Logout revokes the session and disables its shares; stopping a share never deletes the user's file.
8. No global state lock is held during socket I/O, disk I/O, or hashing.
9. Invalid input or peer failure cannot terminate the entire process.

Explicit product assumptions

- Preserve underscore command names and argument order from the PDF. The rendered file-operation page confirms names such as `upload_file` and `show_downloads`; text extraction loses these underscores.
- `upload_file <group_id> <file_path>` uses **group first**, as specified by this assignment. Do not copy another assignment's argument order.
- Support quoted paths containing spaces. Store the display basename separately from the local path.
- A group cannot contain two different contents under the same filename. Same filename and identical metadata adds another seeder; conflicting content is rejected.
- When the owner leaves, transfer ownership to the earliest-accepted remaining member. Leaving and rejoining resets a member's place in that order; pending requests do not qualify. The last member's departure deletes the group and its metadata. Reconstruct join order through deterministic event replay on synchronization/restart.
- Maintain one active session per user. The interim allows a fresh credential login to replace an old client's session after a crash; an already logged-in client must logout before switching accounts. There is no session expiry timer yet.
- Interpret destination as an existing directory; create `<directory>/<filename>`. Refuse accidental overwrites, including duplicate active destinations. Document this interpretation.
- Support empty files: zero pieces, SHA1 of the empty byte stream, immediate verified completion.
- Persist tracker metadata/logs as a recovery implementation choice. Client restart/resume is optional; ordinary peer failures must be recoverable without restarting the client.

CLI command checklist

```
create_user <user_id> <password>
login <user_id> <password>
create_group <group_id>
join_group <group_id>
leave_group <group_id>
list_groups
list_requests <group_id>
accept_request <group_id> <user_id>
logout
upload_file <group_id> <file_path>
list_files <group_id>
download_file <group_id> <file_name> <destination_path>
show_downloads
stop_share <group_id> <file_name>
```

`quit` is required on the tracker console. Client exit may additionally support EOF or a documented convenience command.

6. Custom TCP protocol

Interim implementation: the implemented protocol uses an 8-byte `P2P1`/length header and length-prefixed fields, with one request/response per connection and a 16 MiB limit. Request IDs and statuses live in the payload. The more elaborate header below is a possible final-phase extension, not the current wire format; see README for interoperability details.

Implement and test this before business logic. A TCP `send()` or `recv()` does not correspond to one application message.

Frame format

Field	Size / meaning
Magic	4 bytes identifying this protocol
Version	2 bytes
Message type	2 bytes
Request ID	8 bytes, echoed in response
Payload length	4 bytes, validated before allocation
Status	4 bytes; zero for success/request
Payload	Explicitly encoded fields or piece bytes

Use network byte order and field-by-field serialization. Never send raw C++ structs, pointers, `std::string`, or container memory. Encode strings as a length followed by bytes; encode SHA1 as 20 raw bytes, displaying hexadecimal only for humans. Set a maximum frame size, for example 1 MiB, and paginate/chunk lists and synchronization streams that exceed it.

Message families

- **Client ↔ tracker:** all user/group/file commands, heartbeat, download metadata lookup, share registration, session validation.
- **Peer ↔ peer:** handshake, bitfield request/response, piece request/response, structured error.
- **Tracker ↔ tracker:** hello, replication event, acknowledgement, missing-event request, synchronization barrier/state digest.

Keep each connection's request/response handling sequential initially. Multiple download workers use separate peer connections, avoiding response multiplexing complexity.

Implement `send_all()` and `recv_exact()` with short-I/O handling, `EINTR`, EOF detection, and absolute deadlines. Use nonblocking connect plus `poll()` for bounded connection establishment. Prevent `SIGPIPE` from killing the process using the platform-appropriate mechanism.

Reject malformed lengths, unknown versions/types, invalid piece indexes, unexpected file identity, and incorrect piece payload size. TCP protects transport delivery; SHA1 checks protect file content, and framing validation protects protocol parsing.

Suggested errors: `INVALID_ARGUMENT`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `ALREADY_EXISTS`, `CONFLICT`, `NO_PEERS`, `PIECE_UNAVAILABLE`, `HASH_MISMATCH`, `TIMEOUT`, `RETRY_LATER`, `IO_ERROR`.

Peer requests must identify the group, file, and requester session. Validate membership/session through a tracker when establishing an authorized transfer; cache only for a bounded transfer lifetime. Existing authorized transfers may finish during tracker loss, but new sessions require validation. Document this revocation delay and revalidate periodically. A seeder must also check its own local active-share state before each piece request.

7. Upload and download design

Upload

1. Validate login, group membership, and readable regular file.
2. Read in bounded chunks and update both the current piece hash and whole-file hash.
3. Record one hash per piece; never load the whole file into memory.
4. Check size/metadata again to detect ordinary concurrent file changes.
5. Register metadata and endpoint with the tracker; activate the local share after success.
6. Serve pieces with `pread()` and validate each outgoing piece against its stored hash. Disable/report a changed source rather than sending known-invalid bytes.

Download

1. Obtain immutable file metadata and candidate peers from a tracker.
2. Validate metadata consistency: piece count, size limits, hash counts, final-piece length.
3. Safely create a destination `.part` file; use `ftruncate()` for expected length, without claiming this reserves disk space.
4. Query peer bitfields and initialize the piece scheduler.
5. Workers atomically reserve distinct missing pieces, then release the job lock.
6. Request each assigned piece with a deadline and validate the response's identity/index/length.
7. Verify piece SHA1, write all bytes using a partial-write-safe `pwrite()` loop, then mark verified.
8. Register partial availability once the first piece is verified. Peers obtain current bitfields directly from the client; refresh periodically instead of sending every piece update to trackers.
9. On timeout/disconnection/corruption, release the reservation, record the failed peer for that piece, and retry another eligible peer. Use an attempt generation so late responses cannot overwrite a newer attempt's state.
10. After all pieces pass, stream the complete file through SHA1 and compare the whole-file digest.
11. Finalize to the destination without replacing an existing file; update the local serving path under synchronization and mark complete.

If the final hash fails, keep the job failed/unverified and investigate metadata or disk corruption; never print completion. If no alternate peer has a failed piece, wait with backoff and refresh discovery, then report a stalled/failed state rather than retrying endlessly at full speed.

Scheduling choice

First implementation: sequential missing-piece selection distributed across available peer workers. This already satisfies the assignment if different pieces arrive from multiple peers concurrently.

Final target: choose one initial random available piece, then rarest-first among pieces advertised by connected peers, with random tie-breaking. Complete each assigned piece before choosing another. Rarity is local to the peers discovered by this client, not a global tracker statistic.

Do not implement endgame duplicate requests initially; deadline-based reassignment keeps reservation logic simple.

Rarest-first is an improvement after transfer and recovery work, not a prerequisite for the first usable system.

Resource budget

- Start with 4 download workers **per client in total**, shared fairly across jobs, rather than 4 unbounded workers per file.
- Use round-robin job admission so a large download cannot starve a second file.
- Start with 4 upload workers and a bounded accepted-connection queue.
- Bound peer connections, queued jobs, and outstanding piece buffers.
- Four 512 KiB download buffers use about 2 MiB, excluding upload buffers, metadata, and runtime overhead.
- At 1 GiB, there are 2048 pieces; raw piece hashes occupy 40 KiB and a bitset occupies 256 bytes.

Print completed jobs exactly as `[C] [group_id] filename`. Proposed additional formats: `[D] [group_id] filename 42.3%`, `[W] ... waiting for peers`, and `[F] ... reason`. Compute progress from verified bytes, including the short final piece.

8. Tracker synchronization: design and limits

Interim implementation: healthy forwarding, degraded writes, durable events, request deduplication, and deterministic replay are implemented. For the small interim dataset, synchronization exchanges the full log every approximately 300 ms and around commands, capped at 8 MiB / 50,000 events. Incremental high-water marks, snapshot/compaction, state digests, and peer liveness timers below remain possible final-phase improvements. The tested implementation and its precise limits are documented in README.

Resolve the partition trade-off explicitly

The PDF asks for consistent information at both trackers, continued operation with one tracker, and handling partitions. With only two trackers, a tracker cannot distinguish its partner crashing from their connecting network failing. If both isolated trackers accept changes, they can acknowledge conflicting operations. Immediate consistency and unrestricted writes on both sides of a partition cannot both be guaranteed.

Recommended planning assumption: prioritize availability during disconnection and deterministic convergence after reconnection; provide synchronized results during healthy operation. Record this limitation in the README/report and check whether the instructor instead expects one side to reject writes during a partition. Do not call this design strongly consistent during partitions.

Concrete availability-first design

1. **Healthy mode:** tracker 1 orders mutations. Tracker 2 forwards mutations and fresh-state reads to tracker 1. Tracker 1 serializes each mutation, validates it, appends it to its local log, and replicates it. Return ordinary success after tracker 2 acknowledges application when connected.
2. **Degraded mode:** when the partner is unreachable after bounded retries, a surviving tracker serves from its local state and accepts locally logged operations. Flag responses/logs as degraded. Tracker 1 also continues locally when tracker 2 fails.
3. **Event identity:** each accepted mutation has `(origin_tracker_id, persistent_origin_sequence)`, a Lamport logical timestamp, stable client request ID, actor identity, and deterministic payload. Advance the logical clock on local operations and received events. Persist counters and log entries before acknowledgement.

4. **Replay rules:** apply each event in total order (`logical_timestamp`, `origin_tracker_id`, `origin_sequence`), checking permissions and preconditions against the state produced so far. Earlier valid creation wins; later conflicting creation becomes a recorded rejected event. Logout, leave, deletion, and stop-share are explicit events, never inferred from an absent record.
5. **Recovery:** exchange per-origin contiguous sequence high-water marks, transfer missing events, form the union, and rebuild derived metadata in the same order. Exchange a state digest at an agreed synchronization barrier before returning to healthy mode. Buffer/queue incoming mutations during final catch-up rather than letting clients read a half-rebuilt state.
6. **Conflict consequence:** an operation acknowledged on one isolated side can become invalid after replay. Surface reconciliation conflicts; invalidate affected sessions/shares and require re-login/republishing where needed. Never silently assume union-of-members or last-writer-wins is sufficient for ownership and permissions.
7. **Liveness:** keep peer heartbeat observations separate from durable metadata. Clear/revalidate endpoint liveness after recovery; clients heartbeat/reannounce so restarted trackers do not advertise every historical peer as online.

This is a custom log/replay design, not Raft or a consensus guarantee. It is feasible for three clients but still the highest-risk module. Retain the complete log for the assignment scale; defer log compaction until the basic recovery behavior is proven.

Retry and durability details

- A retry must preserve the same client request ID. In healthy operation, replicated deduplication results prevent the same command taking effect twice.
- A timeout is an **unknown outcome**, not evidence that a write never happened. Reuse the request ID when failing over.
- During disconnection, duplicate request IDs and conflicting results must be deduplicated deterministically during replay; immediate cross-partition exactly-once behavior is not guaranteed.
- Persist log records with lengths/checksums and `fsync()` before reporting durable success. Detect/truncate an incomplete final record after a crash; report corruption in earlier records.
- Keep session tokens/actor information in replicated events so failover does not accidentally authenticate arbitrary callers. Replay historical actor authorization deterministically rather than depending on wall-clock token expiry during rebuild.
- Use monotonic local time for heartbeat deadlines. Convert expiry/revocation decisions into explicit operations when they change durable session state.
- A previously offline tracker must reconcile before advertising recovered metadata as current. If it starts alone, serve in degraded mode with liveness revalidation.

Essential synchronization tests

- Register on tracker 1; log in and query through tracker 2.
- Alternate group/share changes between tracker endpoints and verify equal state after acknowledgement in healthy mode.
- Stop either tracker and continue using the survivor; bring the stopped tracker back and verify recovery.
- Interrupt a response after mutation application; retry the same request ID.
- Partition tracker links, make nonconflicting changes on both sides, reconnect, and verify their union.
- During a partition, create the same user/group independently and issue conflicting ownership/share operations; verify deterministic conflict results and equal final state.
- Restart both trackers from logs; verify metadata recovery and stale peer expiry.

If the instructor requires rejecting conflicting writes rather than reconciling them, redesign degraded write admission around an explicitly fenced writer. Do not claim automatic safe promotion with two isolated nodes and no fencing mechanism.

9. Concurrency and shutdown

- Start with one tracker metadata mutex and serialized mutation application; copy response data under lock and encode/send after releasing it.
- Protect the client's local share registry separately from individual download jobs.
- Define and document lock order; preferably avoid holding two locks simultaneously.
- Use condition variables to wake workers on new jobs, new peers, cancellation, and shutdown.
- Give each descriptor a clear owner using a small move-only RAII wrapper.
- Track in-flight tasks by stable IDs/shared job lifetimes so cancel/delete cannot leave dangling pointers.
- On logout: revoke local sharing immediately, cancel or pause downloads, notify tracker, and clear credentials. A failed tracker notification must not leave the local peer server sharing.
- On shutdown: stop accepting work, signal workers, interrupt blocking socket operations safely, join threads, then release files and state. Avoid closing descriptors from arbitrary threads in ways that race with descriptor reuse.
- Handle read/write/connect failures, invalid paths, allocation failure at sensible boundaries, and disk-full errors with a job/connection error rather than process termination.

10. Implementation schedule and completion gates

Date	Work	Done when
Sep 15	Establish modules/Makefiles, state model, protocol, config parsing, socket helpers	Both binaries start; framed request/response works
Sep 16	All user/group commands, two-tracker synchronization, failover/recovery, tests and interim notes	Both trackers and clients pass the interim suite; submit required interim artifact
Sep 17	Remaining commands and permissions; streaming SHA1; file metadata	Upload/list metadata works; known SHA1 vectors and piece boundaries pass
Sep 18	Peer server, single-peer transfer, <code>.part</code> handling, full verification	Text and binary files download correctly end to end
Sep 19	Shared worker pool, parallel peers, concurrent jobs, progress	Three-client demo proves both kinds of concurrency
Sep 20	Partial seeding, bitfields, retries; rarest-first if baseline is stable	Downloader can seed verified pieces; killed peer does not lose finished pieces
Sep 21	Extend existing replication to file/share metadata	Cross-tracker file discovery and idempotent publication work
Sep 22	Integrate download failover and peer liveness cleanup	Transfers and discovery recover during tracker/peer outages
Sep 23	Extend partition and permission regression to file sharing	Share conflicts converge and authorization remains correct
Sep 24	Full integration tests, 1 GB runs, sanitizer/resource checks	Core acceptance matrix passes; memory and descriptor use stay bounded
Sep 25	Fix remaining defects; finish README/report; rehearse viva/demo	Fresh build and complete demo run without manual repair
Sep 26	Feature freeze, clean-package validation, submit early	ZIP extracts, compiles, runs, and contains required documents

Design replication event fields on day one even though networking/recovery lands later; routing all mutations through one handler early avoids rewriting every command. On September 17, settle partition semantics before building assumptions into the state layer.

Critical dependency chain: framing → authenticated commands → metadata/hashing → correct one-peer transfer → concurrent scheduling → failure recovery → integration and packaging.

If behind schedule, cut rarest-first optimization, block pipelining, persistent client resume, and extra status polish first. Preserve both trackers, integrity, concurrency, permissions, and the required commands.

11. Validation matrix

Use a small C++ test/integration harness or controlled shell orchestration outside the submitted application. The application's prohibition on `system()/exec*()` still applies. Capture commands, expected results, actual results, and relevant logs in test notes.

Area	Cases	Expected result
CLI	Missing arguments, malformed endpoint, unknown command, quoted paths	Clear usage errors; process remains usable
Accounts	Duplicate user, bad password, unauthenticated operation, logout/relogin	Correct status and no unintended state change
Groups	Duplicate group, pending join, non-owner acceptance, nonmember file access, leave	Permissions and membership stay consistent
Files	Empty, 1 byte, 512 KiB – 1, 512 KiB, 512 KiB + 1, multiple pieces, near 1 GB	Correct sizes/hashes and final-piece handling
Content	Text, executable, image, archive, bytes containing NUL	Byte-for-byte identical output
Destinations	Missing directory, permission denied, existing file, same destination twice, disk full	Safe failure; no unrelated file overwritten
Multiple sources	Two seeders of same content and one downloader	Distinct pieces transfer concurrently from both sources
Multiple jobs	One client downloads two files while serving pieces	Both jobs progress; CLI and uploads remain responsive
Corruption	Peer returns modified piece or invalid length	Piece rejected; alternate peer used; no false completion
Peer failure	Kill a seeder mid-piece; leave no peers temporarily	Reservation released; retry/wait status; later recovery
Share control	Stop-share/logout/leave during requests	Subsequent unauthorized serving stops under documented transfer rules
TCP framing	One-byte fragmentation, merged frames, EOF mid-header/payload	Correct decode or bounded failure
Trackers	Both failure directions, retry ambiguity, restart, partition conflicts	Matches the guarantees in Section 8
Resources	Repeated connect/disconnect and download/cancel cycles	No growing thread/descriptor counts or leaks

Run AddressSanitizer/UndefinedBehaviorSanitizer on representative integration tests. Run ThreadSanitizer separately where supported. Compare output using an independent byte comparison as well as the application's SHA1 result, so a shared hash bug cannot make the test falsely pass. Record elapsed transfer time and peak memory for large files; no

throughput target is specified by the PDF.

12. What to use from the BitTorrent paper

Paper idea	Decision for this project
Tracker introduces peers; payload travels directly	Required architectural foundation
Verify pieces before announcing availability	Implement
Downloaders also upload completed pieces	Implement after basic transfer
Random first piece, then rarest-first (§2.4)	Target improvement after baseline scheduler
Sub-piece pipelining (§2.3)	Optional; start with whole 512 KiB piece requests
Endgame duplication/cancellation (§2.4.4)	Defer; use deadlines and reassignment
Tit-for-tat, choking, optimistic unchoking (§3)	Outside required scope; explain in viva, do not prioritize
HTTP tracker and <code>.torrent</code> publishing	Do not adopt; assignment uses custom TCP and group metadata

The paper uses a different typical piece size. Follow the assignment's 512 KB pieces. The assignment also requires a whole-file SHA1 in addition to piece hashes.

13. Final demonstration and submission

Demonstration script

1. Start two trackers and three clients on distinct ports.
2. Create three users; create a group; request and accept membership.
3. Publish a binary file from client A; download it to B and confirm whole-file verification.
4. Download the same file to C from A and B concurrently; show peer/piece IDs in diagnostic logs.
5. Start a second file download on C and show both jobs progressing.
6. Kill one seeder during a transfer; show retry and successful completion where another source exists.
7. Stop a tracker; execute commands on the survivor; restart it and show synchronization.
8. Show stop-share/logout effects, denied nonmember access, and exact completed status output.

Documentation checklist

- [] README: dependencies, Makefiles, configuration format, exact startup and command syntax.
- [] README: architecture, structures, algorithms, wire format, error behavior, assumptions, limitations, tests.
- [] Separate report: implementation approach, synchronization and partition trade-offs, piece selection, protocol rationale, actual challenges/solutions, references.
- [] Evidence: concurrency logs, corruption recovery, tracker recovery/convergence, large-file memory measurements.

[] Viva preparation: explain partial TCP I/O, synchronization semantics, locks, hashing, retries, and scheduler decisions; trace one upload and one download through the code.

[] ZIP named `<Roll_Number>_A3.zip` with required `tracker/`, `client/`, `README.md`, `report`, and every build dependency.

[] No credentials, runtime logs/state, downloaded test files, caches, or unrelated binaries in the submission.

[] Extract to a fresh directory and build/run using only the documented instructions.

Definition of done: every required command works, files are verified at both levels, concurrent multi-peer/multi-file transfers are demonstrated, either tracker can fail and recover, partition behavior is tested and honestly documented, and the extracted submission builds cleanly.